Project Report on

CI/CD for Logistics App

Course Name: DevOps

Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	AKSHAT BAJAJ	EN22CS301088
2	AKANKSHA PARIHAR	EN22CS301079
3	AJAY PAL SINGH THAKUR	EN22CS301077
4	AKSHITA FARKYA	EN22CS301099
5	AASHITA CHAVHAN	EN22CS301018
6	AAKRATI JAIN	EN22CS301013

Group Name: Group 05D1

Project Number: DO -05

Industry Mentor Name: Aashruti Shah

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-26

Problem Statement & Objectives

1. Problem Statement

In modern software development, businesses require fast, secure, and reliable application delivery. Logistics companies especially depend on software applications for shipment tracking, route optimization, inventory management, warehouse operations, delivery scheduling, and customer communication. Any delay or failure in the software system can directly affect business operations and customer satisfaction.

Traditional software deployment methods are mostly manual. Developers build the application manually, run tests separately, and deploy the updated version to the server by hand. This process creates many problems such as:

- Slow release cycles
- Human errors during deployment
- Application downtime
- Inconsistent environments
- Difficulty in rollback
- Delayed bug fixes
- Reduced team productivity

For a Go (Golang)-based Logistics Application, it is important to have an automated system that can continuously integrate code changes, run tests, build the application, create Docker containers, and deploy automatically.

Therefore, this project focuses on creating a **CI/CD Pipeline for Logistics App** that automates the software development lifecycle and ensures faster, safer, and more reliable deployments.

2. Project Objectives

The main objectives of this project are:

- To implement Continuous Integration (CI) for automatic build and testing after every code change.
- To implement Continuous Deployment (CD) for automatic application deployment.
- To reduce manual effort in software release management.
- To improve software quality through automated testing.
- To minimize deployment errors and downtime.
- To containerize the application using Docker.
- To improve collaboration between development and operations teams.
- To deliver updates faster to users.

- To create a scalable DevOps workflow for logistics systems.
- To gain practical knowledge of CI/CD tools and DevOps practices.

3. Scope of the Project

The scope of this project includes:

1. **Technical Scope**
   - Source code version control using GitHub
   - Automated build pipeline
   - Automated test execution
   - Docker image creation
   - Deployment automation
   - Server hosting and release management
2. **Business Scope**

   Useful for:

   - Logistics management companies
   - Shipment tracking systems
   - Warehouse management software
   - Delivery service applications
   - Inventory management platforms
   - E-commerce logistics systems
3. **Future Scope**

   Can be extended to:

   - Cloud deployment (AWS, Azure, GCP)
   - Kubernetes orchestration
   - Monitoring dashboards
   - Security scanning
   - Blue-Green deployment

Proposed Solution

1. Key Features

4. Automated code integration on every commit
5. Continuous Integration (CI) pipeline
6. Continuous Deployment (CD) pipeline
7. Automatic build process for Go application
8. Automated unit and functional testing
9. Docker containerization support

10. Fast and reliable deployment process
11. Reduced manual errors in releases
12. Version control integration with GitHub
13. Real-time build and deployment status
14. Easy rollback to previous stable version
15. Consistent development and production environments
16. Scalable deployment architecture
17. Faster bug fixes and feature releases

2. Overall Architecture / Workflow

The CI/CD Pipeline for the Go (Golang) Logistics Application follows an automated workflow that ensures smooth software development, testing, and deployment. The complete architecture consists of source code management, build automation, testing, containerization, deployment, and monitoring stages.

Workflow Steps:

1. Code Development

Developers write or modify the source code of the Go Logistics Application. New features, bug fixes, or performance improvements are added in the development environment.

2. Version Control Repository

The updated code is pushed to the GitHub repository using Git. GitHub stores the latest source code and maintains version history.

3. CI/CD Pipeline Trigger

Whenever code is pushed to the main branch, the CI/CD pipeline is automatically triggered through GitHub Actions or Jenkins.

4. Build Stage

The pipeline compiles the Go application source code using the Go compiler.

Example Command: go build -o logistics-app

This stage checks for syntax errors and ensures the application builds successfully.

5. Test Stage

Automated tests are executed to verify application functionality

Example Command: go test ./...

This stage ensures that new code changes do not break existing features.

6. Docker Build Stage

After successful testing, a Docker image is created for the application.

Example Command: docker build -t logistics-app

This ensures the application runs consistently across all environments.

7. Deployment Stage

The Docker container is deployed to the production server or cloud platform.

Example Command: docker run -d -p 8080:8080 logistics-app

The updated logistics application becomes available to users.

8. Monitoring & Logs

After deployment, system logs, build reports, and application health are monitored to ensure smooth operation.

<u>Architecture –</u>

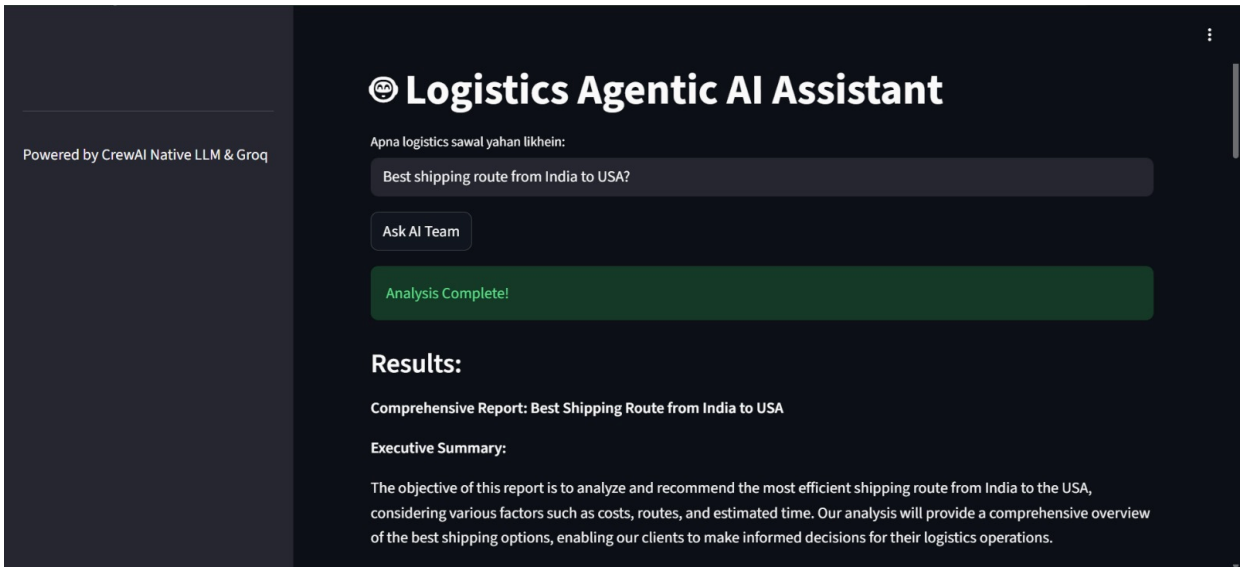
Developer
↓
GitHub Repository
↓
CI/CD Pipeline (GitHub Actions / Jenkins)
↓
Build Stage
↓
Test Stage
↓
Docker Build
↓
Deployment Server
↓
Running Logistics Application
↓
Monitoring & Lo

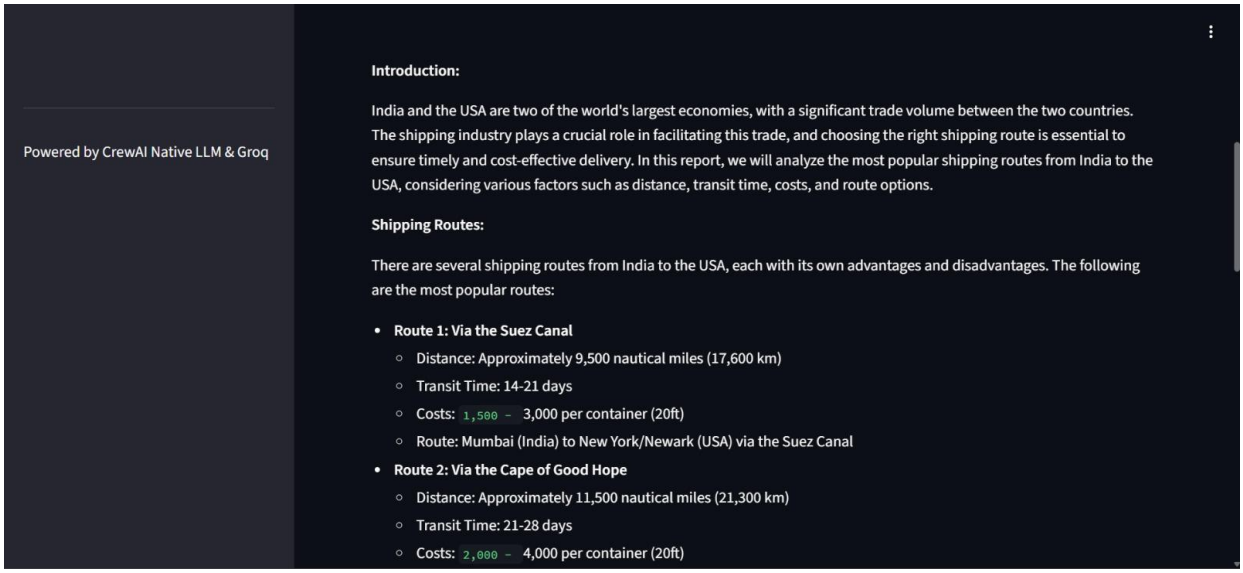
3. Tools & Technologies Used

Tool / Technology	Purpose
Go (Golang)	Backend application development
GitHub	Source code management
Git	Version control
GitHub Actions / Jenkins	CI/CD automation
Docker	Containerization
Linux Server	Deployment environment
YAML	Workflow configuration

Results & Output

1. Screenshots / Outputs





2. Reports / Dashboards / Models

The system generates:

- CI/CD Pipeline Reports
  Detailed reports on build status, test results, and deployment success/failure for each pipeline execution

- Monitoring Dashboards
  Real-time dashboards displaying system metrics such as CPU usage, memory usage, pod status, and application performance

- Log and Error Reports
  Centralized logs and error tracking for debugging issues in application, pipeline, and Kubernetes deployment

- Deployment Insights & Metrics
  Version history, deployment frequency, rollback data, and performance metrics for continuous improvement

3. Key Outcomes

- Fully automated CI/CD pipeline implemented
- Faster release cycles achieved
- Reduced deployment errors

- Improved software quality through testing
- Reliable Docker-based deployment created
- Better team collaboration enabled
- Practical DevOps skills developed
- Scalable system for future expansion created

Conclusion

The CI/CD Pipeline for the Go (Golang) Logistics Application successfully addresses the challenges of traditional software deployment methods. By automating the build, test, and deployment processes, the project ensures faster, more reliable, and error-free software delivery.

The implementation of this pipeline improves software quality, reduces manual effort, and enhances system reliability. It also enables organizations to respond quickly to changing business requirements and deliver updates to users efficiently.

Through this project, valuable knowledge and hands-on experience in DevOps practices, CI/CD tools, Docker, and Go programming have been gained. The project demonstrates how modern technologies can be used to improve the efficiency and scalability of logistics systems

Future Scope & Enhancements

The project can be enhanced further using advanced DevOps practices:

Technical Enhancements

- Kubernetes deployment
- Auto scaling containers
- Blue-Green deployment
- Canary releases
- Auto rollback system

Security Enhancements

- Secret management
- Vulnerability scanning
- Secure image registry
- Role-based access control

Monitoring Enhancements

- Prometheus integration
- Grafana dashboards
- Real-time alerts
- Log analysis tools

Business Enhancements

- Faster feature delivery
- Improved customer satisfaction
- High availability logistics platform
- Better operational efficiency